FastAPI Piscine

Subject booklet. Version 1.0. 6 October 2026.

This booklet contains six independent subjects. Each subject is written to be corrected from this document alone. A corrector does not repair a project, does not rename a file, and does not accept a contract that is "close enough".

The stack against which the contracts below were verified is Python 3.10 or newer, FastAPI 0.142, Pydantic 2.13, and Uvicorn 0.54. Later patch versions are acceptable. A project that depends on a removed interface is not.

Language	Python 3.10 or newer
Framework	FastAPI
Server	Uvicorn
Validation	Pydantic v2, as installed with FastAPI
Form	Six subjects: Day 00 through Day 05
Day 04	Rush. One program. One turn-in.

Common instructions

These rules apply to every subject in this booklet. A subject may add rules. It may not cancel these.

1. The project is written in Python. Type annotations are mandatory on every path operation, every dependency, and every Pydantic model.
2. The only third-party libraries permitted are FastAPI, Pydantic, and Uvicorn. Pydantic is installed as a dependency of FastAPI. The Python standard library is permitted unless an exercise forbids a module.
3. The following names must not be used as module filenames: `fastapi.py`, `uvicorn.py`, `pydantic.py`, `starlette.py`. A local module of that name shadows the installed package.
4. Each exercise is a directory containing a file named `main.py`, except where a subject states otherwise. The ASGI application object in that file is named `app`. The corrector launches it with `uvicorn main:app --reload`. The token `main` is the module. The token `app` is the attribute. `--reload` restarts the process when a source file changes. Reload is mandatory during evaluation. It is not a production setting. A restart discards process memory.
5. If the `uvicorn` executable is not on `PATH`, the equivalent command is `python -m uvicorn main:app --reload`. The corrector may use either, provided the interpreter is the one that has FastAPI installed.
6. The server binds `127.0.0.1` and port `8000` unless a subject says otherwise. Do not pass `--workers`. A multi-worker process model is outside this booklet, and it splits an in-memory store across processes.
7. A path operation may be declared with `def` or with `async def` on Days 00 through 04, provided the HTTP contract holds. Day 05 is the subject that distinguishes the two. Until then, a declaration that does not `await` anything should be a plain `def`. An `async def` that calls a blocking function occupies the event loop. That defect is tolerated before Day 05 only if the graded requests still meet their contracts. It is not tolerated on Day 05.

8. Returning a dictionary, or a Pydantic model, produces a JSON response. A string return is not a JSON object. Where this booklet specifies an object, an object is required.
9. Status codes and payload strings are exact. `healthy` and `Healthy` are different values. Keys are exact. `status` and `state` are different keys.
10. Parsed JSON is the artifact under test. Whitespace in the serialized document is not. The corrector may pretty-print a response. The corrector may not fail a project for the presence or absence of a space after a colon.
11. A `422` response is produced by the framework when validation fails. Its body contains a `detail` list. Each element has a `loc` array. The first entry of `loc` identifies the source: `path`, `query`, or `body`. The corrector grades the status code and `loc`. The human-readable `msg` string is not graded; it varies across Pydantic versions.
12. A request body under test is JSON. The header `Content-Type: application/json` is required. `curl -d` without that header sends a form body. A `422` caused by a form body does not satisfy a schema test, and it does not fail a correct schema. The curl commands in this booklet set the header. The corrector uses those commands.
13. Query strings are part of the URL. The corrector quotes URLs that contain `?`.
14. Internal structure is free: helper functions, modules imported by `main.py`, private names. The HTTP contract is not free. Where a subject requires a mechanism — `HTTPException`, `Depends`, `BackgroundTasks` — the corrector opens the file. A response that merely imitates the status code with another construct fails the exercise.
15. The virtual environment is created once, at the root of the repository, and is not turned in.

```
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
python -m pip install fastapi uvicorn
```

On Windows, the activation command is `venv\Scripts\activate`.

The turned-in repository contains `requirements.txt` at its root, listing `fastapi` and `uvicorn` as direct dependencies. The environment is activated before every evaluation command. `python -c "import fastapi"` must succeed in that shell.

Expected layout:

```
piscine-fastapi/
├── requirements.txt
├── day00/
│   ├── ex00/main.py
│   ├── ex01/main.py
│   └── ex02/main.py
├── day01/
│   ├── ex00/main.py
│   ├── ex01/main.py
│   └── ex02/main.py
├── day02/
│   ├── ex00/main.py
│   ├── ex01/main.py
│   └── ex02/main.py
├── day03/
│   ├── ex00/main.py
│   ├── ex01/main.py
│   └── ex02/main.py
├── day04/
│   └── main.py
└── day05/
    ├── ex00/main.py
    ├── ex01/main.py
    └── ex02/main.py
```

An exercise is validated only when every mandatory item in its checklist passes. A bonus does not repair a failed mandatory item. Refusing the bonus is not a failure. A bonus that breaks a mandatory contract is a failure of the mandatory part.

This booklet does not cover persistence, authentication schemes beyond the teaching key defined herein, deployment, or task queues outside the process. Those topics are out of scope. They are not bonuses unless a subject says so.

Day 00 — Introduction to the ASGI Ecosystem

Assignment name	`day00`
Turn-in directories	`day00/ex00/`, `day00/ex01/`, `day00/ex02/`
Files to turn in	`main.py` in each directory
Allowed libraries	the Python standard library, `fastapi`, `uvicorn`

Mandatory Part

Introduction

An HTTP API in this stack is two programs with a contract between them.

The language runtime does not serve the application. An ASGI server does. ASGI, the Asynchronous Server Gateway Interface, is the contract: the server speaks the network, the application speaks ASGI, and either side can be replaced without rewriting the other. The older Python contract, WSGI, is synchronous. FastAPI requires ASGI.

Uvicorn is the ASGI server used throughout this booklet. FastAPI is the application framework. FastAPI is built on an ASGI toolkit that provides routing, responses, and middleware; that fact explains why the framework can be imported without opening a socket. Constructing a `FastAPI` instance does not bind a port. The process that binds the port is Uvicorn, and it obtains the application by import.

The launch command `uvicorn main:app --reload` is therefore not a script runner. It imports the module `main` and serves the object bound to `app`. A file that is executed as `python main.py` defines that object and then exits, unless the author has added a server call. This subject forbids that shortcut. The corrector starts Uvicorn.

A path operation is a method, a path, and a function. The decorator `@app.get("/")` registers the following function as the handler of `GET /`. The function's return value is the response body. A returned dictionary is encoded as JSON, with media type `application/json`. There is no separate rendering step to invoke for that case.

The framework publishes a generated OpenAPI document at `/openapi.json`, and two renderings of it: Swagger UI at `/docs`, and ReDoc at `/redoc`. No documentation file is turned in. The document is derived from the registered operations and their annotations. On this day the corrector uses `/docs` to execute the graded routes.

`--reload` watches the working directory and restarts the worker after a successful import of the changed source. A syntax error is reported in the server terminal. The browser's connection failure, in that case, is a consequence, not the defect.

General Instructions

- The common instructions of this booklet apply.
- Each exercise is self-contained. Later exercises do not import earlier ones.
- No route other than those required may be necessary for a pass. Additional routes are tolerated only if they do not alter the required contracts and do not cause the server to fail on import.
- The application object is an instance of `FastAPI`, and its name is `app`.
- Do not call a server from inside `main.py`. The corrector will not execute the file as a script.
- Launch from the exercise directory, with the virtual environment activated.

Exercise 00 — Application instance

Turn-in directory: `day00/ex00/`

Title: Application instance

Instructions:

Create a module `main.py` that exposes an ASGI application named `app`.

The application is an instance of `FastAPI`. It has no required path operation. The module must import cleanly under Uvicorn. The generated documentation must be served.

No third-party library other than FastAPI is imported by this module. Uvicorn is the server process, not an import required by `main.py`.

Exercise 01 — Root status route

Turn-in directory: `day00/ex01/`

Title: Root status route

Instructions:

Create a module `main.py` exposing `app`, as in Exercise 00.

Register a path operation for `GET /`.

The success status is `200`. The body is a JSON object with exactly these keys and values:

```
{"status": "operational", "message": "Service is running"}
```

No other key is permitted. The values are exact, including case. The response is a JSON object, not a JSON string.

Exercise 02 — Health route

Turn-in directory: `day00/ex02/`

Title: Health route

Instructions:

Create a module `main.py` exposing `app`.

Register a path operation for `GET /health`.

The success status is `200`. The body is a JSON object with exactly this key and value:

```
{"status": "healthy"}
```

No other key is permitted. This operation reports process availability only. It does not open a file, a socket, or a device. A dependency that can fail must not be added to it.

Bonus Part

The bonus is implemented in `day00/ex02/main.py` and must not change the mandatory body of `GET /health`.

Construct the application with the keyword arguments `title="Piscine API"` and `version="1.0.0"`. Both strings must appear in the document served at `/openapi.json`.

A bonus that renames `app`, or that changes the `/health` contract, fails Exercise 02.

Submission and Peer-evaluation

The corrector activates the environment, changes into the exercise directory, and starts:

```
uvicorn main:app --reload
```

The server terminal must report that the application started, and must not display a traceback. The corrector then uses a second terminal for the commands below, or uses Swagger UI at `http://127.0.0.1:8000/docs`. Both clients are valid. Swagger UI is mandatory for the documentation check: the corrector opens `/docs` in a browser and executes each required operation from that page.

Exercise 00

- [ ] `uvicorn main:app --reload` imports `day00/ex00/main.py` and serves it.
- [ ] `http://127.0.0.1:8000/docs` returns a documentation page, status `200`.
- [ ] `http://127.0.0.1:8000/openapi.json` returns a JSON document, status `200`.
- [ ] The file does not start its own server.

Exercise 01

```
curl -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/
```

- [ ] Status `200`.
- [ ] Parsed body equals `{"status": "operational", "message": "Service is running"}`.
- [ ] The operation is listed in `/docs` and succeeds when executed there.
- [ ] `--reload` is in effect: the corrector may ask the student to confirm the flag. A server started without it fails the subject, even if the payload is correct.

Exercise 02

```
curl -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/health
```

- [ ] Status `200`.
- [ ] Parsed body equals `{"status": "healthy"}`.
- [ ] The operation is listed in `/docs` and succeeds when executed there.
- [ ] Bonus, if claimed: `/openapi.json` contains `Piscine API` and `1.0.0`. If the bonus is not claimed, these strings are not required.

A trailing slash (`/health/`) may produce a redirect. The graded paths have no trailing slash.

> This subject was written by **@khairi**

Day 01 — Routing and Data Validation

Assignment name	`day01`
Turn-in directories	`day01/ex00/`, `day01/ex01/`, `day01/ex02/`
Files to turn in	`main.py` in each directory
Allowed libraries	the Python standard library, `fastapi`, `pydantic`

Mandatory Part

Introduction

A path operation function does not read an unstructured request and then decide what is present. Its signature is the schema. FastAPI inspects the annotations at runtime and binds each parameter from the appropriate part of the request.

The binding rules used this day are the following.

- A parameter whose name appears in the path template, in braces, is a path parameter. `GET /items/{item_id}` and a parameter named `item_id` are the same name. A different spelling is a different parameter.
- A simple parameter whose name does not appear in the path template is a query parameter.
- A parameter whose annotation is a subclass of `pydantic.BaseModel` is read from the JSON body.

Conversion follows the annotation. A path segment is text, because a URL is text. Declaring `item_id: int` requires the framework to parse that text as an integer before the function runs. If parsing or validation fails, the function does not run. The client receives status `422`, and `loc` begins with the source that failed (`path`, `query`, or `body`).

A default value makes a query parameter optional. The absence of a default makes it required. These are different declarations:

- `name: str` — the value is required, and it must be a string.
- `name: str | None` — the value is still required, and JSON `null` is an acceptable value. Absence is not.
- `name: str | None = None` — the key may be omitted. The function then receives `None`.

`str | None` is syntax of Python 3.10 and later, which this booklet already requires.

A Pydantic model is a class. The class body declares fields with annotations. An instance of that class, once the path operation has been entered, is already valid. Field access is by attribute. There is no second validation call to make inside the function for data the signature already accepted.

Unknown fields of a model are ignored under the default configuration. A missing required field, or a value that cannot be accepted as the declared type, is a `422`.

One coercion must not be mistaken for a successful type check. Under the default lax policy, a numeric string in a JSON body may be accepted as a number: the JSON string `"19.5"` can become the float `19.5`. A non-numeric string such as `"free"` cannot. The invalid payloads in this subject are chosen so that they do not depend on coercion. A corrector who substitutes a numeric string and observes a `200` has not found a defect.

Query booleans accept `true`, `false`, `1`, and `0`. A value such as `maybe` is a `422`.

The function is not informed of a validation failure, because it is not called. A trace printed on the first line of the path operation must remain silent when the payload is invalid, and must appear when the payload is valid. That observation is part of the evaluation of Exercise 02.

General Instructions

- The common instructions apply.
- Each exercise is a separate application, launched from its own directory.
- Path parameter names in the signature must match the names in the path template, character for character.
- Models used as body parameters must be subclasses of `BaseModel`. A bare dictionary annotation is not a schema and fails Exercise 02, even if the happy path happens to return JSON.
- Do not install an exception handler that converts validation errors into another status code. `422` is the required status for schema failure.
- Do not read the raw body in order to validate it by hand. Hand-written validation that returns `422` still fails the exercise if the schema is not declared in the signature. The corrector inspects the file.

Exercise 00 — Typed path parameters

Turn-in directory: `day01/ex00/`

Title: Typed path parameters

Instructions:

Implement two path operations.

1. `GET /items/{item_id}`

- `item_id` is an integer path parameter.
- Success status: `200`.
- Body, exactly: `{"item_id": <integer>}`.
- The value is the parsed integer, not a string.
- A non-integer segment produces `422`. It must not produce `404`. Nothing was looked up.

1. `GET /categories/{name}`

- `name` is a string path parameter.
- Success status: `200`.
- Body, exactly: `{"name": "<segment>"}`.
- The segment is returned unchanged.

No store is required. The operations echo the parsed parameters and nothing more.

Exercise 01 — Query parameters with defaults

Turn-in directory: `day01/ex01/`

Title: Query parameters with defaults

Instructions:

Implement `GET /items`.

The operation accepts three query parameters:

Name	Type	Default when the key is omitted
`q`	string or null	`null`
`limit`	integer	`10`
`offset`	integer	`0`

Success status: `200`. The body contains exactly these keys. Integer fields are JSON numbers. `q` is a JSON string when provided, and JSON `null` when omitted.

`GET /items?q=lamp&limit=3&offset=1` therefore returns:

```
{"q": "lamp", "limit": 3, "offset": 1}
```

`GET /items` returns `q` as `null`, `limit` as `10`, and `offset` as `0`.

A non-integer `limit` or `offset` produces `422`, with `loc` identifying the query parameter. The path operation must not run.

Negative values are not rejected on this exercise. They are a bonus, not a mandatory failure.

Exercise 02 — Request body validation

Turn-in directory: `day01/ex02/`

Title: Request body validation

Instructions:

Define a Pydantic model named `Product` with exactly these fields:

Field	Type	Required
`name`	string	yes
`price`	float	yes
`in_stock`	boolean	yes

The JSON names are `name`, `price`, and `in_stock`. No other field is required. No field has a default. Omitting any of the three is a validation error.

Implement `POST /products`. The body parameter is of type `Product`.

On success, the status is `200`, and the body is the validated product: `name` as submitted, `price` as a JSON number equal to the submitted value, `in_stock` as a JSON boolean equal to the submitted value. Additional keys in the response are not permitted.

The following requests must produce `422`, and must not enter the path operation:

- the field `name` is absent;
- `price` is the JSON string `"free"`;
- `in_stock` is the JSON string `"maybe"`.

A well-typed payload must enter the path operation and return `200`. The corrector will require a visible demonstration that the invalid payloads do not enter it: a trace on the first line of the function is sufficient, and it must be removed or left harmless after the demonstration. A trace that changes the HTTP body fails the exercise.

Swagger UI must display the `Product` schema on `POST /products`. If the schema is absent, the parameter is not declared as the model.

Bonus Part

Bonuses are confined to `day01/ex02/main.py` unless noted. They must not weaken a mandatory `422`.

- Declare `price` so that a value less than or equal to zero is rejected with `422`. The mandatory payload `19.5` remains valid.
- Reject unknown body fields with `422` instead of ignoring them.

Either bonus may be taken alone. Both may be taken. A claimed bonus is tested. An unclaimed bonus is not.

Submission and Peer-evaluation

Launch each exercise from its own directory with `uvicorn main:app --reload`. Execute the invalid cases in Swagger UI as well as with curl. The curriculum requires the `422` to be observed in Swagger. A curl-only demonstration fails that item.

Exercise 00

```
curl -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/items/42
curl -sS -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/items/abc
curl -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/categories/tools
```

- [ ] `/items/42` is `200`, and `item_id` is the number `42`.
- [ ] `/items/abc` is `422`, and `loc` identifies the path. It is not `404`.
- [ ] `/categories/tools` is `200`, and `name` is the string `tools`.
- [ ] Both operations appear in `/docs`.

Exercise 01

```
curl -sS "http://127.0.0.1:8000/items"
curl -sS "http://127.0.0.1:8000/items?q=lamp&limit=3&offset=1"
curl -sS -o /dev/null -w "%{http_code}\n" "http://127.0.0.1:8000/items?limit=many"
```

- [ ] The bare request is `200`, with `q` equal to `null`, `limit` equal to `10`, and `offset` equal to `0`.
- [ ] The parameterized request returns the three submitted values, with `limit` and `offset` as numbers.
- [ ] `limit=many` is `422`, and `loc` identifies the query parameter.
- [ ] The function does not run on that `422`. The student demonstrates this if the corrector asks.

Exercise 02

```
curl -sS -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8000/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Lamp","price":19.5,"in_stock":true}'

curl -sS -o /dev/null -w "%{http_code}\n" -X POST http://127.0.0.1:8000/products \
  -H "Content-Type: application/json" \
  -d '{"price":19.5,"in_stock":true}'

curl -sS -o /dev/null -w "%{http_code}\n" -X POST http://127.0.0.1:8000/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Lamp","price":"free","in_stock":true}'

curl -sS -o /dev/null -w "%{http_code}\n" -X POST http://127.0.0.1:8000/products \
```

```
  -H "Content-Type: application/json" \
  -d '{"name":"Lamp","price":19.5,"in_stock":"maybe"}'
```

- [ ] The valid payload is `200` and returns `name` `Lamp`, numeric `price` equal to `19.5`, and boolean `in_stock` `true`.
- [ ] Each of the three invalid payloads is `422`.
- [ ] For the missing name, `loc` contains `body` and `name`. For `"free"`, `loc` contains `body` and `price`. For `"maybe"`, `loc` contains `body` and `in_stock`.
- [ ] None of the invalid payloads enter the path operation. Demonstrated in front of the corrector.
- [ ] Swagger UI shows the `Product` schema, and executing an invalid example from that UI returns `422`.
- [ ] The model is named `Product`.
- [ ] Bonus, if claimed, is tested with `price` `0` and, separately, with an unknown field. Each claimed case is `422`. The mandatory valid payload still passes.

> This subject was written by **@khairi**

Day 02 — HTTP Status Codes and Exception Handling

Assignment name	`day02`
Turn-in directories	`day02/ex00/`, `day02/ex01/`, `day02/ex02/`
Files to turn in	`main.py` in each directory
Allowed libraries	the Python standard library, `fastapi`

Mandatory Part

Introduction

Validation and domain failure are different events, and this subject assigns them different status codes.

Event	Status	Produced by
The request does not match the declared schema	`422`	The framework. The path operation does not run.
The request matches the schema, and the domain rule refuses it	`400` or `404`, as specified	The path operation, by raising `HTTPException`.
An ordinary exception leaves the path operation	`500`	The framework's default handler. This is a defect, not a response strategy.

`HTTPException` is the exception the framework already handles. Raising it aborts the path operation. Code after the raise does not run. The handler writes a JSON object of the form `{"detail": <value>}`. When `detail` is a string, the client observes that string. This subject grades the string.

The constructor takes the numeric status, or a constant from `fastapi.status`. Both are acceptable. The numeric code on the wire is what the checklist records. `status.HTTP_404_NOT_FOUND` is `404`. `status.HTTP_400_BAD_REQUEST` is `400`. `status.HTTP_201_CREATED` is `201`.

The status declared on the decorator is the status of a successful return. It does not apply when an `HTTPException` is raised. `POST` defaults to `200` if the decorator does not say otherwise. Exercise 02 of this day requires `201` on the success path, set explicitly.

An unhandled `KeyError` is not a `404`. A dictionary lookup that raises leaves the function as an ordinary exception, and the client receives `500`. A missing record must be detected and reported with `HTTPException`. Catching `Exception` around that raise, and then returning a dictionary, is also a failure: `HTTPException` is a subclass of `Exception`, and a caught exception does not reach the handler. The client then receives the status of the return, normally `200`.

Dictionary keys keep their type. The integer key `1` and the string key `"1"` are different keys. Path text is converted to an integer only if the parameter is annotated `int`. An unannotated path parameter remains a string, and a lookup against an integer key misses. The stores in this subject are keyed by integers. The path parameters that address them are annotated `int`.

`422` remains the correct response for a path segment that is not an integer. Do not convert that failure into a `404`. The resource was not absent. The URL did not match the schema.

General Instructions

- The common instructions apply.

- Each exercise owns its own store, initialized in that module. Exercises do not import one another.
- The store is process memory. It is not a file. Reload reconstructs it from the source. That is expected.
- Where a `404` or a `400` is required, the file must raise `HTTPException`. A manually assembled response with the same status fails the exercise.
- Do not register a handler that rewrites `HTTPException` into a different shape. The body key is `detail`.
- Seed data, where specified, is exact.

Exercise 00 — In-memory store

Turn-in directory: `day02/ex00/`

Title: In-memory store

Instructions:

Create a module-level dictionary. Its keys are the integers `1` and `2`. Its values are records with exactly these contents:

id	name
1	Keyboard
2	Monitor

The keys of the dictionary are integers, not the strings `"1"` and `"2"`.

Implement `GET /records`.

The success status is `200`. The body is a JSON array of the two records, ordered by ascending `id`. Each record is an object with exactly the keys `id` (a JSON number) and `name` (a JSON string). The array is not wrapped in another object. An implementation that returns a dictionary containing the array fails the exercise.

No other operation is required.

Exercise 01 — Missing resource

Turn-in directory: `day02/ex01/`

Title: Missing resource

Instructions:

Create the same integer-keyed store specified in Exercise 00, inside this exercise's own module.

Implement `GET /records/{id}`.

- `id` is an integer path parameter.
- If the id is present, the status is `200`, and the body is the record, with exactly the keys `id` and `name`.
- If the id is an integer and is not present, raise `HTTPException` with status `404` and `detail` exactly `Record not found`.
- If the segment is not an integer, the response is the framework's `422`. Do not replace it.

The client-facing not-found body is therefore:

```
{"detail": "Record not found"}
```

A `500` on a missing id fails the exercise, including when the response text happens to contain the word "found".

Exercise 02 — Domain rejection

Turn-in directory: `day02/ex02/`

Title: Domain rejection

Instructions:

Create the same store as Exercise 00, inside this module. The store must be mutable for the lifetime of the process.

Implement `POST /records`.

The body is a model, or an equivalent declared schema, with a single required string field, `name`. A missing `name` is `422`, produced by the framework.

Domain rule: names are unique. Comparison is exact and case-sensitive. `Keyboard` and `keyboard` are different names. The seeded names `Keyboard` and `Monitor` are already present.

- If `name` is already present, raise `HTTPException` with status `400` and `detail` exactly `Duplicate record`. Do not insert. Do not use `422` for this case. The schema accepted the request; the domain refused it.
- If `name` is new, store a record under the next integer id and return it. The success status is `201`, set on the operation. The body is the stored record, with exactly the keys `id` and `name`.

The first successful insert in a fresh process receives id `3`, because `1` and `2` are seeded. A rejected insert, whether `400` or `422`, does not consume an id.

A second insert of a name that was itself inserted during the process is also `400`.

Bonus Part

In `day02/ex01/main.py`, add `DELETE /records/{id}`.

- A known id is removed and the status is `204`, with an empty body.
- An unknown integer id is `404`, with `detail` exactly `Record not found`.
- A non-integer segment remains `422`.
- The bonus must not change the mandatory `GET` contract.

If the bonus is not claimed, `DELETE` is not tested.

Submission and Peer-evaluation

Restart the server before testing Exercise 02, so the store is the seed and the next id is `3`.

Exercise 00

```
curl -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/records
```

- [ ] Status `200`.
- [ ] The body is a JSON array of length 2.
- [ ] Index 0 is `{"id": 1, "name": "Keyboard"}`. Index 1 is `{"id": 2, "name": "Monitor"}`.
- [ ] `id` values are numbers.
- [ ] The file contains a dictionary whose keys are integers. The corrector inspects it.

Exercise 01

```
curl -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/records/1
curl -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/records/2
curl -sS -w "\nHTTP %{http_code}\n" http://127.0.0.1:8000/records/99
curl -sS -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/records/abc
```

- [ ] `/records/1` is `200` and the Keyboard record.
- [ ] `/records/2` is `200` and the Monitor record.
- [ ] `/records/99` is exactly `404`, and `detail` is exactly `Record not found`.
- [ ] `/records/abc` is `422`, not `404`.
- [ ] The `404` is implemented with `HTTPException`. Confirmed in the file.
- [ ] A missing key does not produce `500`.
- [ ] Bonus, if claimed: `DELETE /records/1` is `204`; a following `GET /records/1` is `404`; `DELETE /records/99` is `404`.

Exercise 02

```
curl -sS -o /dev/null -w "%{http_code}\n" -X POST http://127.0.0.1:8000/records \
  -H "Content-Type: application/json" \
  -d '{"name":"Keyboard"}'

curl -sS -w "\nHTTP %{http_code}\n" -X POST http://127.0.0.1:8000/records \
  -H "Content-Type: application/json" \
  -d '{"name":"Mouse"}'

curl -sS -o /dev/null -w "%{http_code}\n" -X POST http://127.0.0.1:8000/records \
  -H "Content-Type: application/json" \
  -d '{"name":"Mouse"}'

curl -sS -o /dev/null -w "%{http_code}\n" -X POST http://127.0.0.1:8000/records \
  -H "Content-Type: application/json" \
  -d '{}'
```

- [ ] `Keyboard` is `400`, and `detail` is exactly `Duplicate record`. The store gains no record.
- [ ] `Mouse` is `201`, and the body is `{"id": 3, "name": "Mouse"}`.
- [ ] A second `Mouse` is `400`, with the same detail.
- [ ] A body without `name` is `422`, not `400`.
- [ ] The `400` is implemented with `HTTPException`. Confirmed in the file.
- [ ] The success status is set as `201`, not left at the default `200`.

The corrector may use Swagger UI in place of curl, provided the status code and the parsed body are read off the result. Both clients are valid on this day. The header requirement still applies to curl.

> This subject was written by **@khairi**

Day 03 — Dependency Injection

Assignment name	`day03`
Turn-in directories	`day03/ex00/`, `day03/ex01/`, `day03/ex02/`
Files to turn in	`main.py` in each directory
Allowed libraries	the Python standard library, `fastapi`

Mandatory Part

Introduction

A dependency is a callable that the framework invokes while preparing a path operation. It is declared with `Depends` in a parameter default, or in a `dependencies` list. It is not invoked by the author at decoration time.

The framework resolves the dependency's own parameters by the same rules as a path operation: path, query, body, or a further dependency. The value returned by the dependency is injected into the path operation, if the dependency was declared as a parameter. If the dependency raises, the path operation does not run.

That last sentence is the evaluation criterion of the protected routes on this day. Authentication failure is a domain refusal, reported as `401`. It is not a schema failure. The distinction is mechanical, and it must be understood before the exercises are written.

A parameter declared without a default is required. A required query parameter that is absent fails validation. The client receives `422`, and the dependency callable is never entered. A `raise HTTPException(status_code=401)` placed inside that callable is then unreachable for the missing-key case. The wrong key would still enter the callable, because the key is present and is a string. The two failures would produce two status codes. This subject requires one status code.

The schema must therefore accept an absent key, so that the dependency runs and refuses it. Absence and a wrong value both produce `401`, with the same `detail`. The response must not disclose which of the two checks failed.

`Depends` caches the result of a given dependency for the duration of one request. Two operations that depend on the same callable during one request do not, by themselves, require a hand-written cache. This fact is not exercised by a single dependency on a single operation. It is the reason a dependency may be shared without repeating its body.

Repeating the body is a failure even when the status codes match. The corrector reads the file. The secret value and the comparison against it appear in one function. Each protected operation refers to that function through `Depends`. An operation that inlines the comparison fails the exercise.

ASGI middleware, registered with `add_middleware`, wraps the application as a whole, including requests that match no route. It is the wrong tool for this subject. A project that implements the key check only as middleware, with no `Depends`, fails. Middleware is not forbidden for other purposes. It does not satisfy the dependency requirement.

The teaching credential in this subject is a query parameter. It will appear in access logs and in command history. It exists so that a corrector can call the operation without a client library. It is not a pattern to be copied into a deployed service. The parameter name and the value are fixed below, so that every project is corrected with the same requests.

A dependency that provides pagination follows the same mechanism. Its parameters are query parameters. Its return value is what the path operation uses to slice a sequence. Two operations that require the same window must depend on one function, not on two copies of the arithmetic.

General Instructions

- The common instructions apply.
- The query parameter name is `api_key`.
- The only valid value is `piscine-key`.
- A missing key and any other value, including the empty string, produce status `401` and `detail` exactly `Invalid or missing API key`.
- `422` on a missing key fails the exercise.
- The dependency function that performs this check is named `verify_api_key`.
- The comparison is performed with inequality of values, not with an identity test.
- Do not place `verify_api_key` on the application constructor. An application-wide dependency would also protect `/docs`, and the corrector must be able to open Swagger UI without the key.
- The pagination dependency, where required, is named `pagination`.
- Protected operations increment a module-level counter only when they actually run. This counter is the proof that a `401` did not execute the operation. It is not optional.

Exercise 00 — API key dependency

Turn-in directory: `day03/ex00/`

Title: API key dependency

Instructions:

Implement `verify_api_key` as specified in the general instructions.

Implement `GET /secure`.

- The operation depends on `verify_api_key` through a parameter defaulted with `Depends`. The list form alone is not sufficient for this exercise: the operation must receive the dependency through its signature.
- On success, the status is `200`, and the body is exactly `{"access": "granted"}`.
- The key is not included in the response.

Implement `GET /metrics`, unprotected.

- The status is `200`.
- The body is exactly `{"calls": <integer>}`.
- `calls` starts at `0` when the process starts.
- Each successful `GET /secure` increments `calls` by `1`.
- A `401` from `GET /secure` does not increment `calls`.

A fresh process, one rejected call, and one accepted call, in that order, leaves `calls` equal to `1`.

Exercise 01 — Shared protection

Turn-in directory: `day03/ex01/`

Title: Shared protection

Instructions:

Implement `verify_api_key` once, under that name, with the same contract as Exercise 00.

Protect all three of the following operations with that single dependency. The comparison against `piscine-key` must not appear in the three operations.

Method and path	Success body
`GET /reports`	`{"resource": "reports"}`
`GET /audit`	`{"resource": "audit"}`
`GET /admin`	`{"resource": "admin"}`

Each success status is `200`. Each operation uses `Depends(verify_api_key)`.

Implement unprotected `GET /metrics`, with the same contract as Exercise 00. A successful call to any of the three protected operations increments `calls` by `1`. A `401` on any of them does not.

After a fresh start, the sequence "rejected `/admin`", "accepted `/reports`", "accepted `/audit`" leaves `calls` equal to `2`.

Exercise 02 — Pagination dependency

Turn-in directory: `day03/ex02/`

Title: Pagination dependency

Instructions:

Implement a dependency function named `pagination`.

It reads two query parameters:

Name	Type	Default	Constraint
`skip`	integer	`0`	greater than or equal to `0`
`limit`	integer	`10`	from `1` to `50` inclusive

A missing key uses the default. A value outside the constraint, or a non-integer value, produces `422` and does not enter the path operation. The constraint may be declared on the dependency's parameters. That is sufficient. The path operations must not repeat the range checks.

`pagination` returns the resolved `skip` and `limit` to its caller. Both graded operations obtain the window by depending on `pagination`. Copying the parameters into each operation, without `Depends(pagination)`, fails the exercise.

Provide a fixed sequence of 25 records, with ids `1` through `25` in order. Each record is `{"id": <n>, "label": "item-<n>"}`. The sequence is not modified by requests.

Implement `GET /catalog`.

The success status is `200`. The body is:

```
{
  "skip": 0,
  "limit": 10,
  "total": 25,
  "items": []
}
```

`items` is the slice of the sequence beginning at index `skip` and containing at most `limit` records. `total` is always `25`. `skip` and `limit` in the body are the resolved integers.

Implement `GET /catalog/ids`, depending on the same `pagination` function.

The success status is `200`. The body is:

```
{"skip": 0, "limit": 10, "ids": []}
```

`ids` is the list of `id` values in the same slice, in the same order. This operation exists to prove that the dependency is reused. A second copy of the pagination logic fails the exercise.

Neither operation requires the API key.

Bonus Part

In `day03/ex01/main.py`, also accept the teaching key from the header `X-API-Key`.

- If the query parameter is valid, the request is accepted, whether or not the header is present.
- If the query parameter is absent and the header equals `piscine-key`, the request is accepted.
- If neither is valid, the status is `401`, with the same `detail` as the mandatory contract.
- The query-parameter tests of the mandatory part must still pass.
- The header check lives in `verify_api_key`, not in the three operations.

If the bonus is not claimed, the header is not tested.

Submission and Peer-evaluation

Restart the process before a counter test. Reload, or a previous corrector, will have incremented it.

Exercise 00

```
curl -sS -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/secure
curl -sS http://127.0.0.1:8000/metrics
curl -sS -o /dev/null -w "%{http_code}\n" "http://127.0.0.1:8000/secure?api_key=wrong"
curl -sS http://127.0.0.1:8000/metrics
curl -sS -w "\nHTTP %{http_code}\n" "http://127.0.0.1:8000/secure?api_key=piscine-key"
curl -sS http://127.0.0.1:8000/metrics
```

- [ ] No key: `401`, `detail` exactly `Invalid or missing API key`.
- [ ] After that rejection, `calls` is still `0`.
- [ ] Wrong key: `401`, same `detail`. `calls` is still `0`.
- [ ] Valid key: `200`, body exactly `{"access": "granted"}`.
- [ ] After that success, `calls` is `1`.
- [ ] The file contains one function named `verify_api_key`, used with `Depends` in the signature of `GET /secure`.
- [ ] `GET /secure` does not itself compare the key.
- [ ] `/docs` opens without the key, and the `api_key` parameter is documented on `GET /secure`.

Exercise 01

```
curl -sS -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/admin
curl -sS http://127.0.0.1:8000/metrics
curl -sS -w "\nHTTP %{http_code}\n" "http://127.0.0.1:8000/reports?api_key=piscine-key"
curl -sS -w "\nHTTP %{http_code}\n" "http://127.0.0.1:8000/audit?api_key=piscine-key"
```

```
curl -sS -w "\nHTTP %{http_code}\n" "http://127.0.0.1:8000/admin?api_key=piscine-key"
curl -sS http://127.0.0.1:8000/metrics
```

- [ ] Unauthenticated `/admin` is `401` with the exact detail, and `calls` remains `0`.
- [ ] Each authenticated operation is `200` with its specified body.
- [ ] After the three successes above, `calls` is `3`. The sequence in the instructions (one rejection and two successes) is an alternative test; either sequence is conclusive if the arithmetic matches.
- [ ] The string `piscine-key` appears in `verify_api_key` only. The corrector searches the file.
- [ ] `Depends(verify_api_key)` is attached to all three operations.
- [ ] Bonus, if claimed: a request with no query parameter and header `X-API-Key: piscine-key` is `200`. A request with neither credential is `401`. The mandatory query tests still pass.

The normative counter test, restated so the alternative above cannot be misread: from a fresh process, one `401` and then two authenticated calls yield `calls` equal to `2`. Three authenticated calls with no prior rejection yield `calls` equal to `3`. Both are valid demonstrations. A `401` that increments the counter fails the exercise.

Exercise 02

```
curl -sS "http://127.0.0.1:8000/catalog"
curl -sS "http://127.0.0.1:8000/catalog?skip=20&limit=10"
curl -sS "http://127.0.0.1:8000/catalog/ids?skip=20&limit=10"
curl -sS -o /dev/null -w "%{http_code}\n" "http://127.0.0.1:8000/catalog?limit=51"
curl -sS -o /dev/null -w "%{http_code}\n" "http://127.0.0.1:8000/catalog?skip=-1"
curl -sS -o /dev/null -w "%{http_code}\n" "http://127.0.0.1:8000/catalog?limit=0"
```

- [ ] Bare `/catalog` is `200`, `skip 0`, `limit 10`, `total 25`, and `items` has length 10, beginning with id `1` and label `item-1`.
- [ ] `skip=20&limit=10` returns five items, ids `21` through `25`, in that order.
- [ ] `/catalog/ids?skip=20&limit=10` returns `ids` equal to `[21, 22, 23, 24, 25]`.
- [ ] `limit=51`, `skip=-1`, and `limit=0` are each `422`.
- [ ] Both operations use `Depends(pagination)`. Confirmed in the file. The range checks are not duplicated in the operations.
- [ ] The dependency function is named `pagination`.

> This subject was written by **@khairi**

Day 04 — Rush: Library Management System

Assignment name	`day04`
Turn-in directory	`day04/`
Files to turn in	`main.py`
Allowed libraries	the Python standard library, `fastapi`, `pydantic`
Forbidden	any persistence beyond process memory; any framework other than FastAPI

Mandatory Part

Introduction

This subject is a rush. It has one program and one turn-in. The program is a library catalogue held in process memory. It is corrected as a whole.

The work is the combination of the preceding subjects, not a new framework feature.

- The application is an ASGI object served by Uvicorn, as on Day 00.
- Input is declared with annotations and Pydantic models, as on Day 01. Invalid input is `422`, and the path operation does not run.
- A missing book is `404` by `HTTPException`. A domain collision is `400` by `HTTPException`. An escaped `KeyError` is a `500` and a failure, as on Day 02.
- Mutating operations share one API-key dependency. The list operation shares a pagination dependency. The requirements of Day 03 apply, including the rule that a missing key is `401` and not `422`.

The store is a lie, and the lie is part of the subject. It lives in the process. `--reload` starts a new process and an empty catalogue. A file, a database, or a module that writes records to disk fails the rush, even if every status code is correct. The corrector confirms emptiness by restarting the server before the lifecycle test.

Identifiers are assigned by the server. They are positive integers, beginning at `1`, increasing by one. An identifier is never reused. A rejected request — `401`, `400`, or `422` — does not consume an identifier. A counter that advances only on a successful insert satisfies the rule. Computing one more than the current maximum does not, once deletion exists: deleting the highest id and inserting again would recycle a value a client may still hold.

`PUT` replaces a book. It does not create one. An unknown id is `404`, and the catalogue is unchanged. Omitted optional fields take their defaults, not their previous values. Partial modification is not this operation. It is the bonus.

Titles are unique. The comparison is exact and case-sensitive. Authors are not unique. Two books may name the same author.

The teaching key is the Day 03 credential: query parameter `api_key`, value `piscine-key`, failure status `401`, `detail` exactly `Invalid or missing API key`. It protects `POST`, `PUT`, and `DELETE`. Both read operations are public, so the corrector can observe a catalogue without a credential, and can see which operations declare the dependency. Protecting the reads, or leaving the writes public, fails the rush.

General Instructions

- The common instructions apply.
- The single turn-in file is `day04/main.py`. The object is named `app`.

- The dependency that checks the key is named `verify_api_key`, and it is the only place the credential is compared.
- The dependency that provides `skip` and `limit` is named `pagination`.
- `GET /books` uses `pagination`. The other four operations do not need it.
- Pagination constraints are those of Day 03: `skip` defaults to `0` and must be at least `0`; `limit` defaults to `10` and must be from `1` to `50` inclusive. A violation is `422`.
- The catalogue starts empty. No seed records.
- Do not launch more than one worker.
- Models used for client input do not allow the client to choose `id`. A client-supplied `id` is ignored, unless the bonus that rejects unknown fields is claimed. In the mandatory contract, the server-assigned id is the id that is stored.

Exercise 00 — Library API

Turn-in directory: `day04/`

Title: Library management system

Instructions:

Implement a catalogue of books. A book, as returned by the API, has exactly these fields:

Field	JSON type	Rule
`id`	number	Server-assigned. Not accepted from the client as the stored id.
`title`	string	Required on input. Unique among stored books.
`author`	string	Required on input. Not unique.
`available`	boolean	Optional on input. Default `true`.

An empty `title` or an empty `author` is `422`. A missing required field is `422`.

`POST /books`. Credential required.

- Body fields: `title`, `author`, and optional `available`.
- Success: status `201`, body the stored book, including its new id.
- Duplicate title: status `400`, `detail` exactly `Book already exists`. Nothing is stored. No id is consumed.
- Invalid body: `422`. Nothing is stored. No id is consumed.
- Missing or wrong credential: `401`, with the Day 03 detail. Nothing is stored.

`GET /books`. Public. Uses `pagination`.

- Success: status `200`.
- Body: a JSON array of books, ordered by ascending `id`, after applying `skip` and `limit` to that ordered sequence.
- An empty catalogue is `[]`, status `200`, not `404`.
- The default window is `skip=0`, `limit=10`.

`GET /books/{id}`. Public.

- `id` is an integer path parameter.
- Success: status `200`, and the book.
- Unknown integer: status `404`, `detail` exactly `Book not found`.
- Non-integer segment: `422`.

`PUT /books/{id}`. Credential required.

- Body fields: the same input fields as creation. This is a replacement of `title`, `author`, and `available`. The id does not change.
- A body of `{"title": "Algorithms", "author": "Knuth"}` stores `available` as `true`, even if the previous value was `false`.
- Success: status `200`, and the replaced book.
- Unknown integer: `404`, `detail` exactly `Book not found`. The catalogue is unchanged. The operation must not create a book.
- The replacement title collides with a different stored book: `400`, `detail` exactly `Book already exists`. The target book is unchanged.
- The replacement title is the book's own current title: not a collision. The operation succeeds.
- Invalid body: `422`. The target book is unchanged.
- Missing or wrong credential: `401`. The target book is unchanged.

`DELETE /books/{id}`. Credential required.

- Success: status `200`, body exactly `{"detail": "Book deleted"}`.
- Unknown integer: `404`, `detail` exactly `Book not found`.
- A second deletion of the same id is `404`.
- The deleted id is not reused by a later creation.
- Missing or wrong credential: `401`. The book remains.

All `404` responses in this program use the detail `Book not found`. All duplicate refusals use `Book already exists`. All credential refusals use `Invalid or missing API key`.

Bonus Part

Bonuses must not change the mandatory status codes or the mandatory paths.

- `PATCH /books/{id}`, credential required. The body accepts any non-empty subset of `title`, `author`, and `available`. Omitted fields keep their stored values. An empty object is `400` with `detail` exactly `Empty update`. Unknown id is `404`. A title collision with a different book is `400`. This operation does not replace `PUT`.
- Reject client-supplied unknown fields, including `id`, with `422` on `POST` and `PUT`. If this bonus is claimed, the corrector is told before the test, because a body that includes `id` then fails.

An unclaimed bonus is not tested.

Submission and Peer-evaluation

The corrector restarts Uvicorn so the catalogue is empty and the next id is `1`, then executes the lifecycle. Editing the source during the lifecycle restarts the process under `--reload` and empties the store. The corrector does not edit during the run.

```
BASE=http://127.0.0.1:8000
KEY=piscine-key

curl -sS -w "\nHTTP %{http_code}\n" "$BASE/books"

curl -sS -o /dev/null -w "%{http_code}\n" -X POST "$BASE/books" \
  -H "Content-Type: application/json" \
  -d '{"title":"Algorithms","author":"Knuth"}'

curl -sS -w "\nHTTP %{http_code}\n" -X POST "$BASE/books?api_key=$KEY" \
  -H "Content-Type: application/json" \
  -d '{"title":"Algorithms","author":"Knuth"}'
```

```
curl -sS -w "\nHTTP %{http_code}\n" "$BASE/books"

curl -sS -w "\nHTTP %{http_code}\n" "$BASE/books/1"

curl -sS -o /dev/null -w "%{http_code}\n" -X POST "$BASE/books?api_key=$KEY" \
  -H "Content-Type: application/json" \
  -d '{"title":"Algorithms","author":"Other"}'

curl -sS -w "\nHTTP %{http_code}\n" -X PUT "$BASE/books/1?api_key=$KEY" \
  -H "Content-Type: application/json" \
  -d '{"title":"Algorithms","author":"Knuth","available":false}'

curl -sS -w "\nHTTP %{http_code}\n" "$BASE/books/1"

curl -sS -o /dev/null -w "%{http_code}\n" -X PUT "$BASE/books/99?api_key=$KEY" \
  -H "Content-Type: application/json" \
  -d '{"title":"Missing","author":"Nobody"}'

curl -sS -o /dev/null -w "%{http_code}\n" -X DELETE "$BASE/books/1"

curl -sS -w "\nHTTP %{http_code}\n" -X DELETE "$BASE/books/1?api_key=$KEY"

curl -sS -o /dev/null -w "%{http_code}\n" -X DELETE "$BASE/books/1?api_key=$KEY"

curl -sS -o /dev/null -w "%{http_code}\n" "$BASE/books/1"

curl -sS -w "\nHTTP %{http_code}\n" -X POST "$BASE/books?api_key=$KEY" \
  -H "Content-Type: application/json" \
  -d '{"title":"Compilers","author":"Aho"}'
```

- [ ] The first list is `200` and `[]`.
- [ ] Creation without the key is `401`, with `detail` exactly `Invalid or missing API key`. The list is still empty.
- [ ] Creation with the key is `201`. The book has id `1`, title `Algorithms`, author `Knuth`, and `available` `true`.
- [ ] The list is `200` and contains that book.
- [ ] `GET /books/1` is `200` and the same book.
- [ ] A second creation with the same title is `400`, `detail` exactly `Book already exists`. The list length remains 1. The next id is still unconsumed.
- [ ] The replacement is `200`. Id remains `1`. `available` is `false`. Title and author are unchanged.
- [ ] `PUT` of an unknown id is `404`, `detail` exactly `Book not found`. No book with id `99` appears.
- [ ] Deletion without the key is `401`. `GET /books/1` still succeeds.
- [ ] Deletion with the key is `200`, body exactly `{"detail": "Book deleted"}`.
- [ ] A second deletion is `404`.
- [ ] `GET /books/1` is `404`.
- [ ] The following creation is `201`, and its id is `2`, not `1`.
- [ ] `GET /books/nope` is `422`.
- [ ] `GET /books?limit=51` is `422`.
- [ ] `PUT` with only `title` and `author`, on a book whose `available` is `false`, stores `available` as `true`. The corrector performs this extra request on a fresh book if the lifecycle above has already deleted id `1`. The replacement rule is mandatory.
- [ ] The file uses `HTTPException` for `404` and `400`, `Depends(verify_api_key)` on the three mutating operations, and `Depends(pagination)` on `GET /books`.
- [ ] Reads do not require the key. Confirmed by the unauthenticated list and `GET` in the transcript.

- [ ] Restarting the process yields an empty catalogue. Confirmed by a restart and a final `GET /books`.
- [ ] Swagger UI exposes the five operations. The corrector executes one create, one read, one update of `available`, one delete, and one `GET` of the deleted id from that UI, and observes `201`, `200`, `200`, `200`, and `404` respectively. This walk-through is mandatory in addition to the script.
- [ ] Bonus `PATCH`, if claimed, changes only the supplied fields and rejects an empty object with `400` and `detail` exactly `Empty update`.
- [ ] Bonus strict fields, if claimed, was announced, and a body containing `id` is `422`.

A project that passes the script and fails the Swagger walk-through fails the rush. Both are mandatory.

> This subject was written by **@khairi**

Day 05 — Background Tasks and Concurrency

Assignment name	`day05`
Turn-in directories	`day05/ex00/`, `day05/ex01/`, `day05/ex02/`
Files to turn in	`main.py` in each directory
Allowed libraries	the Python standard library (including `time` and `asyncio`), `fastapi`

Mandatory Part

Introduction

Uvicorn serves the application on an event loop. The loop runs coroutines. It does not run a blocking call concurrently with anything else on that same thread. A blocking call holds the thread until it returns.

FastAPI therefore treats the two declarations differently.

Declaration	Where a path operation runs	A blocking call inside it
`def`	A worker thread, outside the loop	Holds that worker thread. Other requests on the loop can proceed, up to the capacity of the pool.
`async def`	The event loop	Holds the loop. Other requests on that worker wait.

`async` does not make a blocking call asynchronous. It only permits `await` on objects that know how to suspend. `time.sleep` does not suspend. It holds the thread that called it. `asyncio.sleep` suspends, and it can be used only in a coroutine.

The consequence for this day: a function that must call `time.sleep`, or any other blocking library, is a plain `def`. Declaring it `async def` and leaving the blocking call inside it is a defect, including when the call sits in a background task. The client of the request that scheduled the task may already have received a response. The loop then freezes, and the next request waits. A test that measures only the first response will not see the defect. Exercise 01 therefore measures a second request as well.

`BackgroundTasks` schedules callables to run after the response has been sent, in the same process. The parameter is recognized by its annotation, `BackgroundTasks`. The author does not wrap it in `Depends`. The method `add_task` stores a callable and its arguments. It does not call the callable. Passing the result of a call schedules that result, and the call itself has already happened, on the request path, before the response.

Synchronous background callables are executed in a threadpool. Asynchronous background callables are awaited on the loop. Tasks run after the response, in registration order, one after another. They are not a queue. They are not retried. If the process exits, the pending task exits with it. Work that must survive a restart does not belong in this mechanism. This subject is a log line, and a log line is within the limit of the mechanism.

The delay in the graded function is five seconds, produced by `time.sleep(5)`. The log is a file, not a terminal trace. A trace can be missed by a corrector who is looking at the wrong terminal. A file cannot.

General Instructions

- The common instructions apply.
- The blocking function is named `write_log`.

- It is a plain `def`, not an `async def`.
- It calls `time.sleep(5)`, then appends one line to the file `background.log` in the process working directory.
- The line is exactly `task completed`, followed by a newline. No timestamp. No extra text.
- The file is created if it does not exist. Existing content is preserved. Each invocation appends one line.
- The corrector launches Uvicorn from the exercise directory, so `background.log` appears in that directory.
- `print` may be used in addition to the file. The file is the graded evidence. The terminal is not.
- If a write is not visible because of buffering, the implementation must flush the write. A missing line after the full delay is a failure, whatever the cause.
- Do not reduce the delay. A one-second sleep cannot be distinguished reliably from process overhead during evaluation.

Exercise 00 — Blocking task

Turn-in directory: `day05/ex00/`

Title: Blocking task

Instructions:

Implement `write_log` as specified above.

Implement `POST /write-log-sync`.

The operation calls `write_log` directly, on the request path, and only then returns. It does not use `BackgroundTasks`.

The success status is `200`. The body is exactly:

```
{"status": "written"}
```

The operation accepts no body.

When the response is received, `background.log` already contains the new line, and the elapsed time of the request is approximately five seconds. A response faster than four seconds fails the exercise: the sleep did not run on the request path. A response slower than eight seconds fails the exercise: the function is not the specified task.

This exercise exists to make the cost of synchronous work observable. It is not the pattern Exercise 01 implements.

Exercise 01 — Background execution

Turn-in directory: `day05/ex01/`

Title: Background execution

Instructions:

Implement `write_log` again, with the same contract, in this exercise's own module.

Implement `POST /write-log`.

- The operation accepts no body.
- It schedules `write_log` with `BackgroundTasks`.
- It does not call `write_log` itself.

- It returns immediately, with status `200` and body exactly `{"status": "scheduled"}`.
- The new log line is not present in the second that follows the response.
- The new log line is present about five seconds after the response, and in all cases within eight seconds.

Implement `GET /ping`.

- Public. No credential. No body.
- Status `200`. Body exactly `{"status": "ok"}`.
- It performs no sleep and no file write.

A `GET /ping` issued while a scheduled `write_log` is inside its five-second sleep must itself return in well under a second. A ping that waits out the sleep indicates that the blocking call is on the event loop. That fails the exercise, even if `POST /write-log` itself returned quickly. The usual cause is an `async def` that calls `time.sleep`, either as the task or as the path operation.

`BackgroundTasks` must appear in the file, and `add_task` must receive the function `write_log`, not the result of calling it.

Exercise 02 — Declaration comments

Turn-in directory: `day05/ex02/`

Title: Declaration comments

Instructions:

Create a module `main.py` that exposes `app` and imports cleanly.

The file contains at least one path operation declared with `def`, and at least one declared with `async def`. Neither operation calls `time.sleep`, writes `background.log`, or blocks. Each returns a JSON object of the author's choice, with status `200`. The paths must not collide.

Immediately above each operation, comments in the source explain the execution model of that declaration. Elsewhere in the same file, comments explain background tasks. The comments are English, prose, and technically accurate. They are the graded artifact. The corrector reads them. They must cover all of the following claims, in the author's own sentences:

1. A path operation declared with `def` is executed outside the event loop, in a threadpool.
2. A path operation declared with `async def` is executed on the event loop.
3. `time.sleep` inside an `async def` blocks the event loop. The declaration does not make the call non-blocking.
4. `await asyncio.sleep(...)` suspends a coroutine and allows the loop to service other work. It is not interchangeable with `time.sleep`.
5. A synchronous function scheduled with `BackgroundTasks` runs in a threadpool after the response is sent. An asynchronous function scheduled there is awaited on the loop after the response is sent.
6. `BackgroundTasks` does not survive process exit, and it is not a distributed queue.

A comment that contradicts any of these claims fails the exercise, even if the other claims are present. A file that merely lists the six sentences above, with no attachment to a real `def` and a real `async def` in the module, fails the exercise. The comments must sit in this module, next to working declarations.

The module must be valid Python. The corrector imports it by starting Uvicorn. A syntax error fails the exercise.

Bonus Part

In `day05/ex01/main.py`, schedule a second background callable after `write_log`. The second callable appends a line `task archived`, followed by a newline, to the same file, without an additional five-second sleep.

After one request, the file gains exactly two new lines, in this order: `task completed`, then `task archived`. The second line must not appear before the first. The HTTP contract of `POST /write-log` is unchanged.

If the bonus is not claimed, a second line is not required, and an unexpected second line fails the mandatory contract. Do not implement the bonus silently.

Submission and Peer-evaluation

The corrector deletes `background.log` before each exercise, starts Uvicorn from that exercise directory, and issues one untimed request so the process is warm. The timed request is the next one. The first request after a reload is not the measurement.

The warm-up request is a real invocation. On Exercise 00 it already writes a line and consumes five seconds. On Exercise 01 it already schedules a write. The final file therefore contains one line per request issued, including the warm-up. The corrector counts lines per request, not per terminal session. The absence check applies to the interval immediately after the timed response: the line belonging to that response must not be visible yet. A line that appears only because the warm-up's five seconds have elapsed is not, by itself, a failure, provided the timed response itself did not wait.

Exercise 00

```
rm -f background.log
curl -sS -o /dev/null -X POST http://127.0.0.1:8000/write-log-sync
curl -sS -w "\nHTTP %{http_code} in %{time_total}s\n" \
  -X POST http://127.0.0.1:8000/write-log-sync
```

- [ ] The timed response is `200`, the body is exactly `{"status": "written"}`, and `time_total` is greater than 4 and less than 8.
- [ ] When curl returns, `background.log` already contains `task completed`.
- [ ] The file uses `time.sleep(5)` inside `write_log`.
- [ ] `BackgroundTasks` is not used to hide the delay. Confirmed in the file. The delay is on the request path, by requirement of this exercise.

Exercise 01

```
rm -f background.log
curl -sS -o /dev/null -X POST http://127.0.0.1:8000/write-log
curl -sS -w "\nHTTP %{http_code} in %{time_total}s\n" \
  -D - -o /tmp/day05-body.txt -X POST http://127.0.0.1:8000/write-log
sleep 0.3
test ! -f background.log -o -z "$(grep 'task completed' background.log || true)"
curl -sS -w "\nPING %{time_total}s\n" http://127.0.0.1:8000/ping
sleep 6
cat background.log
```

The `test` line is the corrector's check that the log line is still absent a fraction of a second after the response. A failure of that line is a failure of the exercise. The corrector may perform the same observation by reading the file instead of using `test`.

- [ ] The timed `POST` is `200`, the body is exactly `{"status": "scheduled"}`, and `time_total` is under one second. The nominal bar on a quiet machine is under 100 milliseconds. A time on the order of five seconds fails.
- [ ] Within one second of that response, `background.log` does not yet contain the new `task completed` line.
- [ ] `GET /ping`, issued during the delay, is `200`, body exactly `{"status": "ok"}`, and completes in under one second.
- [ ] About five seconds after the `POST`, and before eight seconds have elapsed, the file contains `task completed`.
- [ ] The file schedules `write_log` through `BackgroundTasks.add_task` and does not call `write_log` on the request path.
- [ ] `write_log` is a plain `def`.
- [ ] Stopping Uvicorn during the five seconds prevents the line from being written. The corrector may ask for this demonstration once. It confirms that the task is in-process. Failure to explain that limit, when asked, fails the oral part of this item. The HTTP checklist above is sufficient if the demonstration is refused by time; the corrector then asks the student to state the limit. An incorrect statement fails the item.
- [ ] Bonus, if claimed: the file gains `task archived` as the immediately following line, and the mandatory body is unchanged. If the bonus was not claimed, the file gains exactly one new line per request.

Exercise 02

```
uvicorn main:app --reload
```

- [ ] The server imports the module without a traceback.
- [ ] The file contains a working `def` path operation and a working `async def` path operation. The corrector executes both from `/docs` and receives `200`.
- [ ] Neither operation sleeps or writes `background.log`.
- [ ] Comments adjacent to each declaration, plus comments on background tasks, cover the six mandatory claims.
- [ ] No comment contradicts those claims. In particular, a statement that `async def` runs in a threadpool, or that `time.sleep` is safe on the loop, or that `BackgroundTasks` is a durable queue, fails the exercise.
- [ ] The comments are sentences written against this module, not a detached list with no declarations under them.

The corrector may ask the student to indicate, in the file, the comment that answers a chosen claim. Inability to do so fails the exercise.

> This subject was written by **@khairi**